

Coursework - 2

Due Date: 02/01/2025

Group: S:24-AP1-#003-022

PART I

1.1

Having $\bar{X} = \max_i X_i$

$$\mathbb{E}[\bar{X}] \leq \frac{1}{\lambda} \log \mathbb{E} \left[e^{\lambda \bar{X}} \right] \quad (1.1)$$

$$\Leftrightarrow \lambda \mathbb{E}[\bar{x}] \leq \log \mathbb{E} \left[e^{\lambda \bar{X}} \right] \quad (1.2)$$

$$\Leftrightarrow e^{\lambda \mathbb{E}[\bar{X}]} \leq e^{\log \mathbb{E} \left[e^{\lambda \bar{X}} \right]} = \mathbb{E} \left[e^{\lambda \bar{x}} \right] \quad (1.3)$$

The last inequality is given by the following argument. Consider function $f(x) = e^{\lambda x}$ and given $(\lambda > 0)$ which is convex. Using Jensen inequality, we have

$$f(\mathbb{E}[\bar{X}]) \leq \mathbb{E}[f(\bar{X})]$$

i.e. we get $e^{\lambda \mathbb{E}[\bar{X}]} \leq \mathbb{E} \left[e^{\lambda \bar{X}} \right]$ holds. Hence, the proof is complete.

1.2

the thing we need to show is

$$\frac{1}{\lambda} \log \mathbb{E} \left[e^{\lambda \bar{X}} \right] \leq \frac{1}{\lambda} \log m + \lambda \frac{(b-a)^2}{8}$$

$$\Leftrightarrow \log \mathbb{E} \left[e^{\lambda \bar{X}} \right] \leq \log m + \lambda^2 \frac{(b-a)^2}{8} \quad (1.4)$$

$$\Leftrightarrow \mathbb{E} \left[e^{\lambda \bar{X}} \right] = e^{\log \mathbb{E} \left[e^{\lambda \bar{X}} \right]} \leq m e^{\lambda^2 \frac{(b-a)^2}{8}} \quad (1.5)$$

$$\Leftrightarrow \mathbb{E} \left[e^{\lambda \max_i X_i} \right] \leq \underbrace{e^{\lambda^2(b-a)^2/8} + e^{\lambda^2(b-a)^2/8} + \dots}_m \quad (1.6)$$

From Hoeffding's Lemma we know $\mathbb{E} \left[e^{\lambda X_i} \right] \leq e^{\lambda^2(b-a)^2/8}$ for $\forall i$. Hence,

$$\mathbb{E} \left[e^{\lambda \max_i X_i} \right] = \mathbb{E} \left[\max_i e^{\lambda X_i} \right] \leq \mathbb{E} \left[\sum_{i=1}^m e^{\lambda X_i} \right]$$

Since the maximum of a sequence is always less or equal than the sum of the elements in the sequence. Hence we have

$$\mathbb{E} [e^{\lambda \max_i X_i}] \leq \mathbb{E} \left[\sum_{i=1}^m e^{\lambda X_i} \right] = \sum_{i=1}^m \mathbb{E} [e^{\lambda X_i}] \leq \text{RHS of (1.6)}$$

Hence, the proof is complete.

1.3

The above arguments give us

$$\frac{1}{\lambda} \log \mathbb{E} [e^{\lambda \bar{X}}] \leq \frac{1}{\lambda} \log m + \lambda \frac{(b-a)^2}{8}$$

To make $\frac{1}{\lambda} \log(m) + \lambda \frac{(b-a)^2}{8} \leq \frac{b-a}{2} \sqrt{2 \log(m)}$ We need

$$\log(m) + \lambda^2 \left[\frac{(b-a)^2}{8} \right] - \lambda \left[\frac{b-a}{2} \sqrt{2 \log(m)} \right] \leq 0$$

notice $\Delta = \frac{(b-a)^2}{4} 2 \log(m) - 4 \log(m) \frac{(b-a)^2}{8} = 0$. Hence, only one value can make the inequality holds, which is $\lambda = -\frac{\left(-\frac{b-a}{2} \sqrt{2 \log(a)}\right)}{\frac{2(b-a)^2}{8}} = \frac{2 \sqrt{2 \log(a)}}{b-a}$.

1.4

We now have $\mathbb{E} [\max_i X_i] \leq \frac{b-a}{2} \sqrt{2 \log(m)}$. And we know

$$R(s) = \mathbb{E}_\sigma \left[\max_{x \in S} \left(\frac{1}{n} \sum_{j=1}^n \sigma_j x_j \right) \right]$$

I hope to use the inequality given in question 1.3 to derive our new inequality, but by studying some examples I found that this is not a possible way. The old inequality is too loose to give the new one. Hence, we use the idea given above to drive the new one.

We want to show that

$$\mathcal{R}(S) = \mathbb{E}_\sigma \left[\max_{x \in S} \frac{1}{n} \sum_{j=1}^n \sigma_j x_j \right] \leq \max_{x \in S} \|x\|_2 \frac{\sqrt{2 \log m}}{n},$$

where $\sigma_1, \dots, \sigma_n$ are i.i.d. Rademacher random variables.

Step 1: Converting the expectation to the form. Define

$$\bar{X} = \max_{x \in S} \frac{1}{n} \sum_{j=1}^n \sigma_j x_j.$$

By a standard inequality in question 1.1, we have

$$\mathbb{E}_\sigma[\bar{X}] \leq \frac{1}{\lambda} \log \mathbb{E}_\sigma[e^{\lambda \bar{X}}] \quad \text{for any } \lambda > 0.$$

Hence,

$$\mathcal{R}(S) = \mathbb{E}_\sigma[\bar{X}] \leq \frac{1}{\lambda} \log \mathbb{E}_\sigma \left[\exp \left(\lambda \max_{x \in S} \frac{1}{n} \sum_{j=1}^n \sigma_j x_j \right) \right].$$

Since the exponential function is monotonic, we can replace $\max_{x \in S}$ *inside* the exponential by an overall sum:

$$\max_{x \in S} \exp \left(\frac{\lambda}{n} \sum_{j=1}^n \sigma_j x_j \right) \leq \sum_{x \in S} \exp \left(\frac{\lambda}{n} \sum_{j=1}^n \sigma_j x_j \right).$$

Thus

$$\mathbb{E}_\sigma \left[\max_{x \in S} e^{\frac{\lambda}{n} \sum_{j=1}^n \sigma_j x_j} \right] \leq \mathbb{E}_\sigma \left[\sum_{x \in S} e^{\frac{\lambda}{n} \sum_{j=1}^n \sigma_j x_j} \right] = \sum_{x \in S} \mathbb{E}_\sigma \left[e^{\frac{\lambda}{n} \sum_{j=1}^n \sigma_j x_j} \right].$$

Hence, we obtain:

$$\frac{1}{\lambda} \log \left(\sum_{x \in S} \mathbb{E}_\sigma \left[e^{\frac{\lambda}{n} \sum_{j=1}^n \sigma_j x_j} \right] \right).$$

Step 2: Factorizing the expectation using independence. Because $\sigma_1, \dots, \sigma_n$ are i.i.d. Rademacher variables,

$$\mathbb{E}_\sigma \left[e^{\frac{\lambda}{n} \sum_{j=1}^n \sigma_j x_j} \right] = \mathbb{E}_\sigma \left[\prod_{j=1}^n e^{\frac{\lambda}{n} \sigma_j x_j} \right] = \prod_{j=1}^n \mathbb{E}_\sigma \left[e^{\frac{\lambda}{n} \sigma_j x_j} \right].$$

This separates the sum in the exponent into a product of n identical forms of expectation.

Step 3: Applying Hoeffding's Lemma. Hoeffding's Lemma in question 1.2 tells us that if Y is a zero-mean random variable with $Y \in [a, b]$ almost surely, then

$$\mathbb{E}[e^{tY}] \leq \exp \left(\frac{t^2(b-a)^2}{8} \right).$$

In our setting, $\sigma_j x_j$ has mean 0 (since σ_j has mean 0) and $\sigma_j x_j \in [-|x_j|, |x_j|]$. Taking $t = \frac{\lambda}{n}$ and $b - a = 2|x_j|$, we get:

$$\mathbb{E}_\sigma \left[e^{\frac{\lambda}{n} \sigma_j x_j} \right] \leq \exp \left(\frac{(\frac{\lambda}{n})^2 (2|x_j|)^2}{8} \right) = \exp \left(\frac{\lambda^2 x_j^2}{2n^2} \right).$$

Then, multiplying these bounds across all $j = 1, \dots, n$ yields:

$$\prod_{j=1}^n \exp \left(\frac{\lambda^2 x_j^2}{2n^2} \right) = \exp \left(\frac{\lambda^2}{2n^2} \sum_{j=1}^n x_j^2 \right) = \exp \left(\frac{\lambda^2 \|x\|_2^2}{2n^2} \right).$$

Step 4: Summation over S and extracting a maximum. We then have

$$\sum_{x \in S} \mathbb{E}_\sigma \left[e^{\frac{\lambda}{n} \sum_{j=1}^n \sigma_j x_j} \right] \leq \sum_{x \in S} \exp\left(\frac{\lambda^2 \|x\|_2^2}{2n^2}\right) \leq m \max_{x \in S} \exp\left(\frac{\lambda^2 \|x\|_2^2}{2n^2}\right).$$

Thus,

$$\frac{1}{\lambda} \log \left[\sum_{x \in S} \mathbb{E}_\sigma [\cdot] \right] \leq \frac{1}{\lambda} \log \left[m \max_{x \in S} \exp\left(\frac{\lambda^2 \|x\|_2^2}{2n^2}\right) \right] = \frac{1}{\lambda} \log m + \max_{x \in S} \frac{\lambda \|x\|_2^2}{2n^2}.$$

Hence,

$$\mathcal{R}(S) \leq \frac{1}{\lambda} \log m + \max_{x \in S} \frac{\lambda \|x\|_2^2}{2n^2}.$$

Step 5: Optimizing over λ . To minimize

$$f(\lambda) = \frac{1}{\lambda} \log m + \max_{x \in S} \frac{\lambda \|x\|_2^2}{2n^2}$$

with respect to $\lambda > 0$, we set the derivative to zero:

$$-\frac{1}{\lambda^2} \log m + \max_{x \in S} \frac{\|x\|_2^2}{2n^2} = 0 \implies \lambda = \max_{x \in S} \frac{n \sqrt{2 \log m}}{\|x\|_2}.$$

Step 6: Substitution of λ . Substitute $\lambda = \frac{n \sqrt{2 \log m}}{\|x\|_2}$ back into the expression:

$$\frac{1}{\lambda} \log m = \frac{\|x\|_2}{n \sqrt{2 \log m}} \log m, \quad \frac{\lambda \|x\|_2^2}{2n^2} = \frac{\|x\|_2 \sqrt{2 \log m}}{2n}.$$

Both terms are of the order $\frac{\|x\|_2 \sqrt{2 \log m}}{n}$, so combining them appropriately and taking the maximum over $x \in S$ yields an overall upper bound

$$\mathcal{R}(S) \leq \max_{x \in S} \|x\|_2 \frac{\sqrt{2 \log m}}{n}.$$

This is the desired result.

1.5

With the same idea and intuition in mind, it is easy to give the required inequality following the similar steps as above.

We want to bound the Rademacher complexity

$$R_S(\mathcal{H}) = \mathbb{E}_\sigma \left[\max_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i) \right],$$

where $\mathcal{H}$ is a finite hypothesis class and $S = (x_i)_{i=1}^n$ is our sample.

Step 1: Exponential Bound

A common approach to bound the expectation of a maximum is to use exponential-type inequalities such as the one in above questions. Applying Jensen's inequality to the function $x \mapsto e^x$, we get

$$\mathbb{E}_\sigma \left[\max_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i) \right] \leq \frac{1}{\lambda} \log \mathbb{E}_\sigma \left[\exp \left(\lambda \max_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i) \right) \right].$$

We then rewrite the inner exponential:

$$\exp \left(\lambda \max_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i) \right) = \max_{f \in \mathcal{H}} \exp \left(\frac{\lambda}{n} \sum_{i=1}^n \sigma_i f(x_i) \right).$$

Hence,

$$\mathbb{E}_\sigma \left[\exp \left(\lambda \max_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i) \right) \right] = \mathbb{E}_\sigma \left[\max_{f \in \mathcal{H}} \prod_{i=1}^n \exp \left(\frac{\lambda}{n} \sigma_i f(x_i) \right) \right].$$

Since the famous $\max(A) \leq \sum(A)$ for any set A , we get

$$\max_{f \in \mathcal{H}} \prod_{i=1}^n \exp \left(\frac{\lambda}{n} \sigma_i f(x_i) \right) \leq \sum_{f \in \mathcal{H}} \prod_{i=1}^n \exp \left(\frac{\lambda}{n} \sigma_i f(x_i) \right).$$

Taking expectations and then the logarithm:

$$\frac{1}{\lambda} \log \left(\sum_{f \in \mathcal{H}} \mathbb{E}_\sigma \left[\prod_{i=1}^n \exp \left(\frac{\lambda}{n} \sigma_i f(x_i) \right) \right] \right).$$

Therefore,

$$\mathbb{E}_\sigma \left[\max_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i) \right] \leq \frac{1}{\lambda} \log \left(\sum_{f \in \mathcal{H}} \mathbb{E}_\sigma \left[\prod_{i=1}^n \exp \left(\frac{\lambda}{n} \sigma_i f(x_i) \right) \right] \right).$$

Step 2: Bounding the Exponential Product

Next, consider

$$\mathbb{E}_\sigma \left[\prod_{i=1}^n \exp \left(\frac{\lambda}{n} \sigma_i f(x_i) \right) \right].$$

Using the inequality $\mathbb{E}_\sigma[e^{\alpha\sigma}] \leq e^{\alpha^2/2}$ for a Rademacher random variable σ and a constant α , we get:

$$\mathbb{E}_\sigma \left[\prod_{i=1}^n \exp \left(\frac{\lambda}{n} \sigma_i f(x_i) \right) \right] \leq \prod_{i=1}^n \exp \left(\frac{\lambda^2}{2n^2} f(x_i)^2 \right) = \exp \left(\frac{\lambda^2}{2n^2} \sum_{i=1}^n f(x_i)^2 \right).$$

Step 3: Summation over the Class

Plugging this bound back in, we have

$$\sum_{f \in \mathcal{H}} \mathbb{E}_\sigma \left[\prod_{i=1}^n \exp\left(\frac{\lambda}{n} \sigma_i f(x_i)\right) \right] \leq \sum_{f \in \mathcal{H}} \exp\left(\frac{\lambda^2}{2n^2} \sum_{i=1}^n f(x_i)^2\right).$$

Since

$$\sum_{f \in \mathcal{H}} \exp\left(\frac{\lambda^2}{2n^2} \sum_{i=1}^n f(x_i)^2\right) \leq |\mathcal{H}| \max_{f \in \mathcal{H}} \exp\left(\frac{\lambda^2}{2n^2} \sum_{i=1}^n f(x_i)^2\right),$$

we find

$$\frac{1}{\lambda} \log \left(|\mathcal{H}| \max_{f \in \mathcal{H}} \exp\left(\frac{\lambda^2}{2n^2} \sum_{i=1}^n f(x_i)^2\right) \right) = \frac{1}{\lambda} \log |\mathcal{H}| + \max_{f \in \mathcal{H}} \frac{\lambda}{2n^2} \sum_{i=1}^n f(x_i)^2.$$

Step 4: Optimize λ

We now have

$$\frac{1}{\lambda} \log |\mathcal{H}| + \max_{f \in \mathcal{H}} \frac{\lambda}{2n^2} \sum_{i=1}^n f(x_i)^2.$$

Let

$$\max_{f \in \mathcal{H}} \sum_{i=1}^n f(x_i)^2 = \sum_{i=1}^n (f^*(x_i))^2,$$

for some $f^* \in \mathcal{H}$. Taking the derivative with respect to λ , setting it to zero, and solving gives

$$\lambda^* = \frac{n \sqrt{2 \log |\mathcal{H}|}}{\sqrt{\sum_{i=1}^n (f^*(x_i))^2}}$$

Substituting λ^* into our expression leads to the overall bound

$$\frac{1}{n} \sqrt{2 \log |\mathcal{H}| \max_{f \in \mathcal{H}} \sum_{i=1}^n f(x_i)^2}.$$

Hence, for a finite class $\mathcal{H}$, the Rademacher complexity is controlled by

$$\max_{f \in \mathcal{H}} \frac{1}{n} \sqrt{2 \log |\mathcal{H}| \sum_{i=1}^n f(x_i)^2}$$

PART II

2.1

The probability of mis-classification for a binary classification rule $c : \mathcal{X} \rightarrow \mathcal{Y}$, $\mathcal{Y} = \{-1, 1\}$, is defined as:

$$R(c) = \mathbb{P}_{(x,y) \sim \rho}(c(x) \neq y), \quad (1.7)$$

where ρ is the joint distribution of the input-output pair (x, y) . Note that by definition we have

$$R(c) = \mathbb{P}_{(x,y) \sim \rho}(c(x) \neq y) = \mathbb{E}_{(x,y) \sim \rho} [\mathbf{1}_{\{c(x) \neq y\}}] \quad (1.8)$$

We can express the expectation as an integral with respect to the measure $\rho(x, y)$ over the domain $\mathcal{X} \times \mathcal{Y}$:

$$R(c) = \int_{\mathcal{X} \times \mathcal{Y}} \mathbf{1}_{\{c(x) \neq y\}} d\rho(x, y) \quad (1.9)$$

2.2

(a) Squared Loss

The squared loss is

$$\ell(f(x), y) = (f(x) - y)^2.$$

Its expected risk can be written as

$$\mathcal{E}(f) = \int_{\mathcal{X}} \left[\int_{\mathcal{Y}} (f(x) - y)^2 d\rho(y | x) \right] d\rho_{\mathcal{X}}(x).$$

Focusing on each fixed x , we aim to minimize

$$\int_{\mathcal{Y}} (f(x) - y)^2 d\rho(y | x).$$

When $y \in \{-1, +1\}$, we can write

$$\mathcal{E}(f) = \int_{\mathcal{X}} \left[(f(x) - 1)^2 \rho(y = 1 | x) + (f(x) + 1)^2 \rho(y = -1 | x) \right] d\rho_{\mathcal{X}}(x).$$

Taking the derivative with respect to $f(x)$ and setting it to zero yields

$$f_*(x) = \rho(y = 1 | x) - \rho(y = -1 | x).$$

Hence, the Bayes-optimal predictor for squared loss is the difference of the conditional probabilities of the two classes.

(b) Exponential Loss

$$\ell(f(x), y) = e^{-y f(x)}.$$

The expected risk is

$$\mathcal{E}(f) = \int_{\mathcal{X}} \left[e^{-f(x)} \rho(y = 1 | x) + e^{f(x)} \rho(y = -1 | x) \right] d\rho_{\mathcal{X}}(x).$$

Again, minimize pointwise in x by differentiating with respect to $f(x)$ and setting to zero:

$$\frac{\partial}{\partial f(x)} \left(e^{-f(x)} \rho(y = 1 | x) + e^{f(x)} \rho(y = -1 | x) \right) = 0.$$

Solving for $f(x)$ yields

$$f_*(x) = \frac{1}{2} \log \frac{\rho(y = 1 | x)}{\rho(y = -1 | x)}.$$

This is half the log odds of the two classes.

(c) Logistic Loss

The logistic loss is

$$\ell(f(x), y) = \log(1 + e^{-y f(x)}).$$

The expected risk becomes

$$\mathcal{E}(f) = \int_{\mathcal{X}} \left[\log(1 + e^{-f(x)}) \rho(y = 1 | x) + \log(1 + e^{f(x)}) \rho(y = -1 | x) \right] d\rho_{\mathcal{X}}(x).$$

Differentiating with respect to $f(x)$ and setting to zero leads to

$$e^{f(x)} \rho(y = -1 | x) = \rho(y = 1 | x),$$

so

$$f_*(x) = \log \frac{\rho(y = 1 | x)}{\rho(y = -1 | x)}.$$

This is precisely the log-odds function, which is well known as the Bayes-optimal solution under logistic loss.

(d) Hinge Loss

Finally, consider the hinge loss,

$$\ell(f(x), y) = \max(0, 1 - y f(x)).$$

Its expected risk is

$$\mathcal{E}(f) = \int_{\mathcal{X}} \left[\max(0, 1 - f(x)) \rho(y = 1 | x) + \max(0, 1 + f(x)) \rho(y = -1 | x) \right] d\rho_{\mathcal{X}}(x).$$

We analyze three scenarios for $f(x)$:

Case 1: $f(x) \geq 1$ The term $1 - f(x)$ is non-positive, so its hinge contribution for $y = 1$ is 0. The remaining loss term is $(1 + f(x)) \rho(y = -1 | x)$. Minimizing w.r.t. $f(x)$ in this region shows its derivative is nonnegative, suggesting the best point in that region is $f_*(x) = 1$.

Case 2: $f(x) \leq -1$ Here, the term $\max(0, 1 + f(x))$ vanishes for $f(x) \leq -1$. The integrand simplifies and yields derivative $-\rho(y = 1 | x) \leq 0$. This indicates the minimum is at $f_*(x) = -1$.

Case 3: $-1 < f(x) < 1$ In this interval, the hinge loss from each class is

$$(1 - f(x)) \rho(y = 1 | x) + (1 + f(x)) \rho(y = -1 | x).$$

Differentiating and setting to zero gives

$$\rho(y = 1 | x) - \rho(y = -1 | x) = 0,$$

which implies

$$\rho(y = 1 | x) = \rho(y = -1 | x).$$

Equivalently, if $\rho(y = 1 | x) \geq \rho(y = -1 | x)$, then the optimal $f(x)$ in that region is $+1$. Otherwise, it is -1 .

Putting it all together, the hinge-loss minimizer is essentially the sign of $\rho(y = 1 | x) - \rho(y = -1 | x)$; specifically:

$$f_*(x) = \begin{cases} +1, & \text{if } \rho(y = 1 | x) \geq \rho(y = -1 | x), \\ -1, & \text{otherwise.} \end{cases}$$

Summary

We have a brief summary here.

- **Squared loss:**

$$f_*(x) = \rho(y = 1 | x) - \rho(y = -1 | x).$$

- **Exponential loss:**

$$f_*(x) = \frac{1}{2} \log \frac{\rho(y = 1 | x)}{\rho(y = -1 | x)}.$$

- **Logistic loss:**

$$f_*(x) = \log \frac{\rho(y = 1 | x)}{\rho(y = -1 | x)}$$

- **Hinge loss:**

$$f_*(x) = \text{sign}(\rho(y = 1 | x) - \rho(y = -1 | x))$$

2.3

We measure how often a classifier c gets things wrong by looking at its *mis-classification error*, $R(c)$. Which is

$$R(c) = \int_{\mathcal{X} \times \mathcal{Y}} \mathbf{1}_{\{c(x) \neq y\}} d\rho(x, y),$$

which in another way

$$R(c) = \int_{\mathcal{X}} \left(\int_{\mathcal{Y}} \mathbf{1}_{\{c(x) \neq y\}} d\rho(y | x) \right) d\rho_{\mathcal{X}}(x).$$

Breaking that down further, since y can only be $+1$ or -1 ,

$$R(c) = \int_{\mathcal{X}} \left(\mathbf{1}_{\{c(x) \neq 1\}} \rho(y = 1 | x) + \mathbf{1}_{\{c(x) \neq -1\}} \rho(y = -1 | x) \right) d\rho_{\mathcal{X}}(x).$$

Follow the below intuition:

1. **Case $c(x) = +1$:** If our classifier predicts $+1$, it makes a mistake whenever the true label is -1 . The probability of that happening (given x) is $\rho(y = -1 | x)$.
2. **Case $c(x) = -1$:** If our classifier predicts -1 , it is wrong whenever the true label is $+1$. The probability for that scenario is $\rho(y = +1 | x)$.

To minimize the overall error, we want to choose whichever label is more likely under the conditional distribution of y given x . This leads us to the so-called Bayes decision rule:

$$c^*(x) = \begin{cases} +1, & \text{if } \rho(y = 1 | x) \geq \rho(y = -1 | x), \\ -1, & \text{otherwise.} \end{cases}$$

The above decision rule achieves the smallest possible mis-classification error under ρ .

2.4

Surrogate frameworks in problem (2.2) are fisher consistent if there is such function d described in the question.

a) Squared Loss

Under squared loss, we have:

$$f_*(x) = \rho(y = 1 | x) - \rho(y = -1 | x).$$

- If $\rho(y = 1 | x) \geq \rho(y = -1 | x)$, then $f_*(x) \geq 0$.
- If $\rho(y = 1 | x) < \rho(y = -1 | x)$, then $f_*(x) < 0$.

Hence, the classification rule that minimizes misclassification error is given by

$$d(f_*(x)) = \text{sign}(f_*(x)) = \begin{cases} +1 & \text{if } f_*(x) \geq 0, \\ -1 & \text{otherwise.} \end{cases} \quad (1.10)$$

b) Exponential Loss

For exponential loss, the minimizer is

$$f_*(x) = \frac{1}{2} \log\left(\frac{\rho(y=1|x)}{\rho(y=-1|x)}\right).$$

- If $\rho(y = 1 | x) \geq \rho(y = -1 | x)$, then $\frac{\rho(y=1|x)}{\rho(y=-1|x)} \geq 1$, so $\log\left(\frac{\rho(y=1|x)}{\rho(y=-1|x)}\right) \geq 0$, implying $f_*(x) \geq 0$.
- If $\rho(y = 1 | x) < \rho(y = -1 | x)$, then $f_*(x) < 0$.

Thus, the derived classification rule is

$$d(f_*(x)) = \text{sign}(f_*(x)) = \begin{cases} +1 & \text{if } f_*(x) \geq 0, \\ -1 & \text{otherwise.} \end{cases} \quad (1.11)$$

c) Logistic Loss

For logistic loss, the solution satisfies

$$f_*(x) = \log\left(\frac{\rho(y=1|x)}{\rho(y=-1|x)}\right).$$

This is again positive (or zero) if $\rho(y = 1 | x) \geq \rho(y = -1 | x)$ and negative otherwise. Therefore, the same classification rule applies:

$$d(f_*(x)) = \text{sign}(f_*(x)) = \begin{cases} +1 & \text{if } f_*(x) \geq 0, \\ -1 & \text{otherwise.} \end{cases} \quad (1.12)$$

d) Hinge Loss

We again have

$$d(f_*(x)) = \text{sign}(f_*(x)) = \begin{cases} +1 & \text{if } f_*(x) \geq 0, \\ -1 & \text{otherwise.} \end{cases} \quad (1.13)$$

which is obvious.

Summary

In all four cases (squared, exponential, logistic, and hinge losses), the Fisher consistency hold.

2.5

2.5.1

We begin with the mis-classification error:

$$R(\text{sign}(f)) = \int_{\mathcal{X} \times \mathcal{Y}} \mathbf{1}_{\{\text{sign}(f(x)) \neq y\}} d\rho(x, y).$$

Splitting over the two possible labels $y = 1$ and $y = -1$, we obtain

$$R(\text{sign}(f)) = \int_{\mathcal{X}} \left[\mathbf{1}_{\{\text{sign}(f(x)) \neq +1\}} \rho(y = 1 | x) + \mathbf{1}_{\{\text{sign}(f(x)) \neq -1\}} \rho(y = -1 | x) \right] d\rho_{\mathcal{X}}(x)$$

We want to compare this to $R(\text{sign}(f_*))$. Consider the difference:

$$\begin{aligned} \left| R(\text{sign}(f)) - R(\text{sign}(f_*)) \right| &= \left| \int_{\mathcal{X}} \left[\rho(y = 1 | x) (\mathbf{1}_{\{\text{sign}(f(x)) \neq +1\}} - \mathbf{1}_{\{\text{sign}(f_*(x)) \neq +1\}}) \right. \right. \\ &\quad \left. \left. + \rho(y = -1 | x) (\mathbf{1}_{\{\text{sign}(f(x)) \neq -1\}} - \mathbf{1}_{\{\text{sign}(f_*(x)) \neq -1\}}) \right] d\rho_{\mathcal{X}}(x) \right| \end{aligned}$$

Note that if $\text{sign}(f(x)) = \text{sign}(f_*(x))$, then each difference in the indicator functions is zero:

$$\mathbf{1}_{\{\text{sign}(f(x)) \neq +1\}} - \mathbf{1}_{\{\text{sign}(f_*(x)) \neq +1\}} = 0,$$

$$\mathbf{1}_{\{\text{sign}(f(x)) \neq -1\}} - \mathbf{1}_{\{\text{sign}(f_*(x)) \neq -1\}} = 0.$$

Hence, the only region contributing to the integral is

$$\mathcal{X}_f = \{x \in \mathcal{X} : \text{sign}(f(x)) \neq \text{sign}(f_*(x))\}.$$

Restricting the integral to $\mathcal{X}_f$ yields

$$\begin{aligned} &\left| \int_{\mathcal{X}_f} \left[\rho(y = 1 | x) (\mathbf{1}_{\{\text{sign}(f(x)) \neq +1\}} - \mathbf{1}_{\{\text{sign}(f_*(x)) \neq +1\}}) \right. \right. \\ &\quad \left. \left. + \rho(y = -1 | x) (\mathbf{1}_{\{\text{sign}(f(x)) \neq -1\}} - \mathbf{1}_{\{\text{sign}(f_*(x)) \neq -1\}}) \right] d\rho_{\mathcal{X}}(x) \right| \end{aligned}$$

Within $\mathcal{X}_f$, there are two possible cases:

- $\text{sign}(f(x)) = +1$ and $\text{sign}(f_*(x)) = -1$,
- $\text{sign}(f(x)) = -1$ and $\text{sign}(f_*(x)) = +1$.

For the first case, the difference in the integrand is $\rho(y = 1 | x) - \rho(y = -1 | x)$. For the second, it is $-\rho(y = 1 | x) + \rho(y = -1 | x)$.

Hence, the integral becomes

$$\left| \int_{\mathcal{X}_f} \mathbf{1}_{\{\text{sign}(f_*(x)) = +1\}} [\rho(y = 1 | x) - \rho(y = -1 | x)] + \mathbf{1}_{\{\text{sign}(f_*(x)) = -1\}} [-\rho(y = 1 | x) + \rho(y = -1 | x)] d\rho_{\mathcal{X}}(x) \right|$$

Since

$$f_*(x) = \rho(y = 1 \mid x) - \rho(y = -1 \mid x),$$

we have

$$\text{sign}(f_*(x)) = \begin{cases} +1, & \text{if } \rho(y = 1 \mid x) \geq \rho(y = -1 \mid x), \\ -1, & \text{otherwise.} \end{cases}$$

Thus, each part of the integrand in above integral is nonnegative, allowing us to remove the absolute value:

$$\int_{\mathcal{X}_f} \mathbf{1}_{\{\text{sign}(f_*(x))=+1\}} |\rho(y = 1 \mid x) - \rho(y = -1 \mid x)| + \mathbf{1}_{\{\text{sign}(f_*(x))=-1\}} |\rho(y = 1 \mid x) - \rho(y = -1 \mid x)| d\rho_{\mathcal{X}}(x).$$

Noting that $|\rho(y = 1 \mid x) - \rho(y = -1 \mid x)| = |f_*(x)|$, this becomes

$$\int_{\mathcal{X}_f} |f_*(x)| d\rho_{\mathcal{X}}(x).$$

Therefore,

$$\left| R(\text{sign}(f)) - R(\text{sign}(f_*)) \right| = \int_{\mathcal{X}_f} |f_*(x)| d\rho_{\mathcal{X}}(x).$$

2.5.2

By definition,

$$\mathcal{X}_f = \{x \in \mathcal{X} : \text{sign}(f(x)) \neq \text{sign}(f_*(x))\}.$$

If $\text{sign}(f_*(x)) = +1$, it means $f_*(x) \geq 0$ while $f(x) < 0$. Hence,

$$|f_*(x) - f(x)| = |f_*(x)| + |f(x)| \geq |f_*(x)|.$$

Similarly, when $\text{sign}(f_*(x)) = -1$, we have $f_*(x) < 0$ and $f(x) \geq 0$. Then

$$|f_*(x) - f(x)| = -f_*(x) + f(x) = |f_*(x)| + |f(x)| \geq |f_*(x)|.$$

Thus, on $\mathcal{X}_f$,

$$|f_*(x) - f(x)| \geq |f_*(x)|.$$

Integrating over $\mathcal{X}_f$ gives

$$\int_{\mathcal{X}_f} |f_*(x)| d\rho_{\mathcal{X}}(x) \leq \int_{\mathcal{X}_f} |f_*(x) - f(x)| d\rho_{\mathcal{X}}(x).$$

Since

$$\int_{\mathcal{X}_f} |f_*(x) - f(x)| d\rho_{\mathcal{X}}(x) \leq \int_{\mathcal{X}} |f_*(x) - f(x)| d\rho_{\mathcal{X}}(x),$$

we can use the concavity of $\sqrt{\cdot}$ (Jensen's inequality):

$$\int_{\mathcal{X}} \sqrt{|f_*(x) - f(x)|^2} d\rho_{\mathcal{X}}(x) = \mathbb{E}_{\rho_{\mathcal{X}}}[\sqrt{|f_*(x) - f(x)|^2}] \leq \sqrt{\mathbb{E}_{\rho_{\mathcal{X}}} [|f_*(x) - f(x)|^2]}.$$

2.5.3

Next, let us consider

$$\mathcal{E}(f) - \mathcal{E}(f_*),$$

where

$$\mathcal{E}(f) = \int_{\mathcal{X}} \int_{\mathcal{Y}} (f(x) - y)^2 d\rho(y | x) d\rho_{\mathcal{X}}(x),$$

and similarly for $\mathcal{E}(f_*)$. We can write

$$\mathcal{E}(f) - \mathcal{E}(f_*) = \int_{\mathcal{X}} \int_{\mathcal{Y}} [(f(x) - y)^2 - (f_*(x) - y)^2] d\rho(y | x) d\rho_{\mathcal{X}}(x).$$

Since $\mathcal{Y} = \{+1, -1\}$, expanding and grouping terms gives

$$\int_{\mathcal{X}} [f(x)^2 - f_*(x)^2 - 2f(x)(\rho(y=1|x) - \rho(y=-1|x)) + 2f_*(x)(\rho(y=1|x) - \rho(y=-1|x))] d\rho_{\mathcal{X}}(x).$$

Noting that

$$f_*(x) = \rho(y=1|x) - \rho(y=-1|x),$$

we substitute to get

$$\int_{\mathcal{X}} [f(x)^2 - f_*(x)^2 - 2f(x)f_*(x) + 2f_*(x)^2] d\rho_{\mathcal{X}}(x),$$

which simplifies to

$$\int_{\mathcal{X}} [f(x)^2 - 2f(x)f_*(x) + f_*(x)^2] d\rho_{\mathcal{X}}(x) = \int_{\mathcal{X}} (f(x) - f_*(x))^2 d\rho_{\mathcal{X}}(x).$$

Thus,

$$\mathcal{E}(f) - \mathcal{E}(f_*) = \int_{\mathcal{X}} |f(x) - f_*(x)|^2 d\rho_{\mathcal{X}}(x) = \mathbb{E}_{\rho_{\mathcal{X}}} [|f(x) - f_*(x)|^2].$$

Summary

Combining the above three sections and fisher consistency of binary classification with squared loss, we have the desired comparison inequality.

PART III

1

One-Versus-Rest is a technique that enables models designed for binary classification to tackle multiclass classification problems. Instead of building a single model to distinguish among all classes simultaneously, OvR decomposes the problem into multiple binary classifiers, each responsible for distinguishing one class from all other classes. Suppose we have a classification problem with K distinct classes. OvR involves creating K separate binary classifiers. Each classifier is trained to recognize one specific class (the “one”) against all the other classes combined (the “rest”).

- Classifier 1: Class 1 vs. Not Class 1
- Classifier 2: Class 2 vs. Not Class 2
- ...
- Classifier K : Class K vs. Not Class K

During the training phase, each binary classifier learns to distinguish its target class from the rest. This process is repeated for each class, resulting in K distinct models. When making predictions on new, unseen data, we apply all classifiers: The input data is passed through all K classifiers, each producing a score indicating the likelihood of the input belonging to that class. Then we select the highest score: The class with the highest score among the K classifiers is selected as the final prediction.

2

We process the data sequentially and iteratively in the number of epochs.

Suppose we have m train data and there are K classes. We first convert labels to K classification labels. For each classification, assign $+1$ if the label is that class, otherwise -1 . This will create a $K \times m$ label matrix.

We create a matrix of α with size $K \times m$. Each row of α corresponds to an OvR classifier. We treat $\mathbf{w}_t^T \mathbf{x}_t = 0$ as mis-classification. Since the initialization is 0, the first column of α is initialized to be first column of $\mathbf{y}$.

When processing the training data, we compute dot product from α and t -th column of kernel gram matrix. The dot product of them corresponds to $\sum_{i=1}^{t-1} \alpha_i K(\mathbf{x}_i, \mathbf{x}_t)$ for each K . The results are compared with classification labels by checking $\mathbf{w}_t \mathbf{y}_t$. We update the weight as long as $\mathbf{w}_t \mathbf{y}_t \leq 0$ and put the updated weights into t -th column of α . After iterating through all the data for 1 time, we fill up the whole α matrix. Starting from second epoch, we update α by adding the updated weights (if any) onto the position corresponding to same training $\mathbf{x}_t$.

To choose ideal epochs, we can partition the data into training and testing set and run on different epochs and select smallest testing error. We did on full data with 80% training data and 20% testing data with polynomial kernel order 2. We run 10 times each and plot the averaged errors. As we increased the number of epochs, the training errors and testing errors decreased and then leveled off. This means we have extracted all the information we can get from training data since a certain number of epochs. While perceptrons can overfit, the risk is somewhat mitigated by the discrete update rules and the fact they only update on mis-classifications. Overfitting can still happen if the data is noisy and we keep adjusting for outliers repeatedly. But in our data set, it looks not oscillating. We choose **15** epochs for later training.

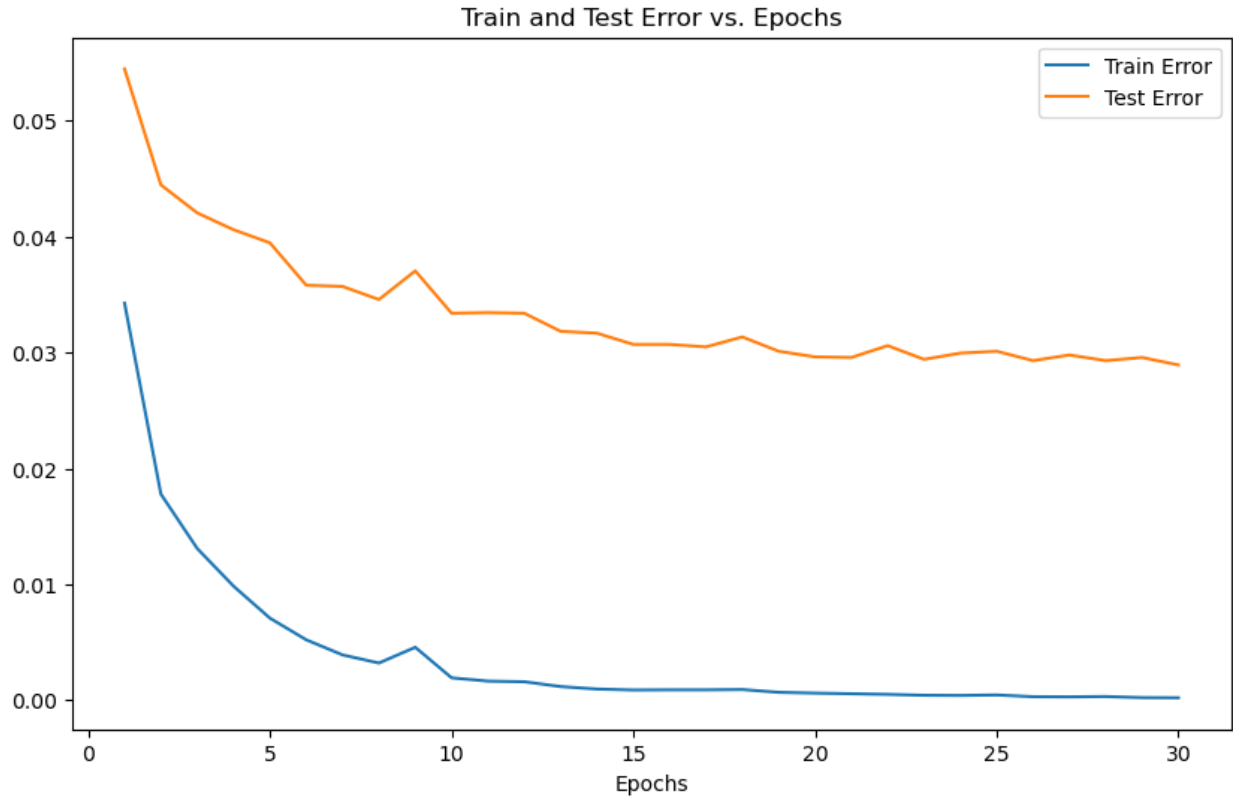


Figure 1.1. Training and Testing Errors v.s. Epochs

3

Order 4 polynomial kernel provides best training and testing errors. Training errors can be very small but testing errors keep stable around 2.7%. Provided that we have 9298 data with 80% training and 20% testing. We have 7438 training data and 1860 test data. 2.7% error rate in testing means we only misclassified 50 data. For the training error 0.0147%, we only misclassified 1 data point.

d	Training	Testing
1	0.0610514 ± 0.00899232	0.0894086 ± 0.0085467
2	$0.00118984 \pm 0.000991866$	0.0316667 ± 0.00439711
3	$0.000242001 \pm 0.000109223$	0.0279839 ± 0.00325469
4	$0.000147889 \pm 8.39607e-05$	0.0269892 ± 0.00370773
5	$0.000194945 \pm 0.000187621$	0.0277957 ± 0.00361497
6	$0.000188223 \pm 0.000177345$	0.0273925 ± 0.00338101
7	$0.000215112 \pm 0.000423874$	0.0284946 ± 0.00348012

Table 1.1. Training Error v.s. Testing Error for Polynomial Kernel

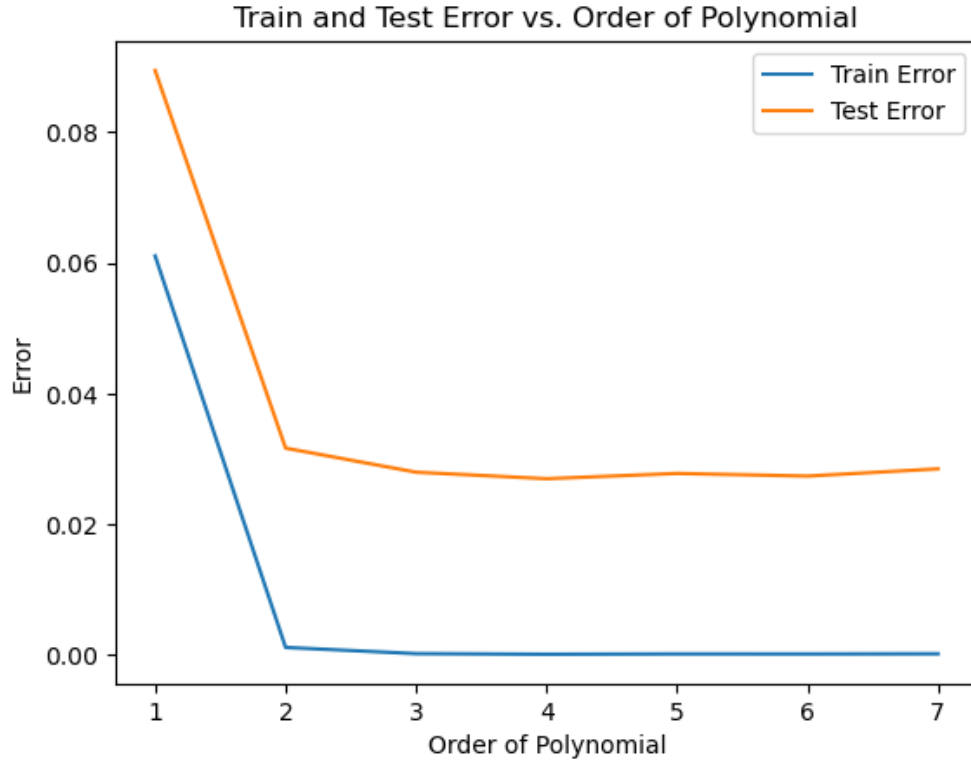


Figure 1.2. Training Error v.s. Testing Error for Polynomial Kernel

4

We have very low training error, with around 6 data misclassified. Testing error is similar with Q3, with 54 data points misclassified. The optimal d^* over 20 runs are:

[4, 5, 4, 6, 5, 4, 5, 5, 7, 3, 3, 6, 5, 5, 4, 3, 5, 3, 4, 4]

The optimal order of polynomial is around 4 or 5, which is consistent with what we have seen in Q3.

Metric	Mean $\pm$ Std
d^*	4.5 ± 1.07238
Training	$0.000786502 \pm 0.00272538$
Testing	0.029086 ± 0.00541091

5

The confusion matrix is presented as below. The row represents the true label and the columns are mislabeled ones.

We also present mean rate in a formatted form. The highest mis-classification rate pair is (3, 5), (5, 3), (8, 0), (8, 3), (8, 5) with more than 1% error rate. This is expected as these numbers are harder to distinguish.

	0	1	2	3	4	5	6	7	8	9
0	0.00% $\pm$ 0.00%	0.08% $\pm$ 0.17%	0.19% $\pm$ 0.24%	0.15% $\pm$ 0.22%	0.07% $\pm$ 0.13%	0.18% $\pm$ 0.24%	0.23% $\pm$ 0.20%	0.02% $\pm$ 0.07%	0.05% $\pm$ 0.11%	0.11% $\pm$ 0.18%
1	0.00% $\pm$ 0.00%	0.00% $\pm$ 0.00%	0.10% $\pm$ 0.21%	0.04% $\pm$ 0.16%	0.86% $\pm$ 2.88%	0.00% $\pm$ 0.00%	0.07% $\pm$ 0.19%	0.18% $\pm$ 0.27%	0.12% $\pm$ 0.22%	0.02% $\pm$ 0.08%
2	0.33% $\pm$ 0.36%	0.13% $\pm$ 0.28%	0.00% $\pm$ 0.00%	0.81% $\pm$ 0.66%	0.46% $\pm$ 0.61%	0.15% $\pm$ 0.32%	0.17% $\pm$ 0.26%	0.41% $\pm$ 0.42%	0.29% $\pm$ 0.32%	0.05% $\pm$ 0.23%
3	0.19% $\pm$ 0.29%	0.13% $\pm$ 0.33%	0.63% $\pm$ 0.55%	0.00% $\pm$ 0.00%	0.06% $\pm$ 0.19%	1.97% $\pm$ 1.46%	0.00% $\pm$ 0.00%	0.26% $\pm$ 0.32%	0.93% $\pm$ 0.63%	0.19% $\pm$ 0.33%
4	0.03% $\pm$ 0.12%	0.80% $\pm$ 0.59%	0.74% $\pm$ 0.60%	0.08% $\pm$ 0.19%	0.00% $\pm$ 0.00%	0.12% $\pm$ 0.29%	0.39% $\pm$ 0.64%	0.25% $\pm$ 0.33%	0.06% $\pm$ 0.17%	0.84% $\pm$ 0.87%
5	0.95% $\pm$ 0.74%	0.07% $\pm$ 0.21%	0.38% $\pm$ 0.46%	1.35% $\pm$ 1.08%	0.47% $\pm$ 0.60%	0.00% $\pm$ 0.00%	1.03% $\pm$ 0.81%	0.07% $\pm$ 0.21%	0.43% $\pm$ 0.42%	0.31% $\pm$ 0.34%
6	0.95% $\pm$ 0.74%	0.27% $\pm$ 0.41%	0.38% $\pm$ 0.38%	0.03% $\pm$ 0.13%	0.42% $\pm$ 0.47%	0.44% $\pm$ 0.46%	0.00% $\pm$ 0.00%	0.00% $\pm$ 0.00%	0.18% $\pm$ 0.28%	0.10% $\pm$ 0.34%
7	0.03% $\pm$ 0.13%	0.29% $\pm$ 0.48%	0.38% $\pm$ 0.51%	0.36% $\pm$ 0.43%	0.78% $\pm$ 0.70%	0.25% $\pm$ 0.36%	0.00% $\pm$ 0.00%	0.00% $\pm$ 0.00%	0.40% $\pm$ 0.57%	0.60% $\pm$ 0.50%
8	1.09% $\pm$ 0.80%	0.47% $\pm$ 0.64%	0.60% $\pm$ 0.78%	1.27% $\pm$ 0.95%	0.65% $\pm$ 0.49%	1.06% $\pm$ 0.81%	0.11% $\pm$ 0.26%	0.40% $\pm$ 0.59%	0.00% $\pm$ 0.00%	0.32% $\pm$ 0.35%
9	0.12% $\pm$ 0.24%	0.03% $\pm$ 0.13%	0.10% $\pm$ 0.24%	0.09% $\pm$ 0.20%	0.94% $\pm$ 0.56%	0.24% $\pm$ 0.34%	0.00% $\pm$ 0.00%	0.83% $\pm$ 0.99%	0.17% $\pm$ 0.26%	0.00% $\pm$ 0.00%

Table 1.2. Confusion Matrix

Mean	0	1	2	3	4	5	6	7	8	9
0	0.00%	0.08%	0.19%	0.15%	0.07%	0.18%	0.23%	0.02%	0.05%	0.11%
1	0.00%	0.00%	0.10%	0.04%	0.86%	0.00%	0.07%	0.18%	0.12%	0.02%
2	0.33%	0.13%	0.00%	0.81%	0.46%	0.15%	0.17%	0.41%	0.29%	0.05%
3	0.19%	0.13%	0.63%	0.00%	0.06%	1.97%	0.00%	0.26%	0.93%	0.19%
4	0.03%	0.80%	0.74%	0.08%	0.00%	0.12%	0.39%	0.25%	0.06%	0.84%
5	0.95%	0.07%	0.38%	1.35%	0.47%	0.00%	1.03%	0.07%	0.43%	0.31%
6	0.95%	0.27%	0.38%	0.03%	0.42%	0.44%	0.00%	0.00%	0.18%	0.10%
7	0.03%	0.29%	0.38%	0.36%	0.78%	0.25%	0.00%	0.00%	0.40%	0.60%
8	1.09%	0.47%	0.60%	1.27%	0.65%	1.06%	0.11%	0.40%	0.00%	0.32%
9	0.12%	0.03%	0.10%	0.09%	0.94%	0.24%	0.00%	0.83%	0.17%	0.00%

Table 1.3. Confusion Matrix: Mean only

For standard deviation, pair (1, 4) has the largest standard deviation, also (3, 5), (5, 3), (9, 7). This is expected as well since they are easy to mis-classified.

6

We ran 100 times of classifier. Each run we did cross validation to choose best order d^* and test on full data (9298). We count the misclassifications of each index. The top misclassified labels and misclassified counts are:

Index	Misclassification Count (Out of 100)
5296	98
8261	52
7524	30
2377	30
6356	28
7413	28
3227	28
7630	28
8835	27

Table 1.5. Top Hard-to-predict Labels

We plot the first 8 since there are ties on the 5th.

Mean	0	1	2	3	4	5	6	7	8	9
0	0.00%	0.17%	0.24%	0.22%	0.13%	0.24%	0.20%	0.07%	0.11%	0.18%
1	0.00%	0.00%	0.21%	0.16%	2.88%	0.00%	0.19%	0.27%	0.22%	0.08%
2	0.36%	0.28%	0.00%	0.66%	0.61%	0.32%	0.26%	0.42%	0.32%	0.23%
3	0.29%	0.33%	0.55%	0.00%	0.19%	1.46%	0.00%	0.32%	0.63%	0.33%
4	0.12%	0.59%	0.60%	0.19%	0.00%	0.29%	0.64%	0.33%	0.17%	0.87%
5	0.74%	0.21%	0.46%	1.08%	0.60%	0.00%	0.81%	0.21%	0.42%	0.34%
6	0.74%	0.41%	0.38%	0.13%	0.47%	0.46%	0.00%	0.00%	0.28%	0.34%
7	0.13%	0.48%	0.51%	0.43%	0.70%	0.36%	0.00%	0.00%	0.57%	0.50%
8	0.80%	0.64%	0.78%	0.95%	0.49%	0.81%	0.26%	0.59%	0.00%	0.35%
9	0.24%	0.13%	0.24%	0.20%	0.56%	0.34%	0.00%	0.99%	0.26%	0.00%

Table 1.4. Confusion Matrix: Std only

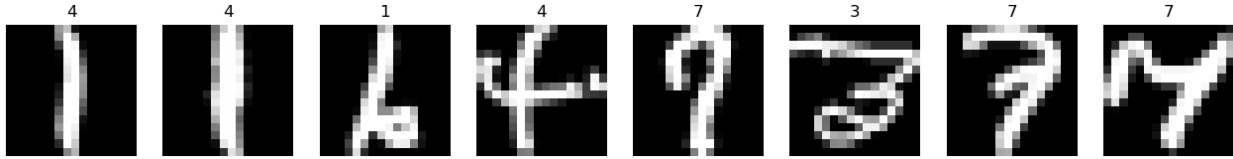


Figure 1.3. Hardest-to-predict Data

It is not surprising that these data are hardest to predict as first 3 pixel images are completely mess - they are mislabeled. The rest are also difficult to distinguish as the writings are not natural.

7

(a)

For Gaussian kernel, we have:

- Small c ($c \rightarrow 0$): The kernel becomes wide (similar to a linear kernel). Every pair of points has a significant similarity, leading to smoother decision boundaries.
- Large c ($c \rightarrow +\infty$): The kernel becomes narrow. Only very close points are considered similar, leading to more complex decision boundaries that can capture intricate patterns but may overfit the data.

Each data is 16×16 with values scaled between -1 and 1 . The largest value $\|\mathbf{p} - \mathbf{q}\|^2$ can take is 2^{10} . Since this is the maximally different data, we would like to make the kernel between them close to 0. We can choose $\exp(-c \cdot 2^{10}) = \exp(-2^3)$, which suggests $c = 2^{-7}$. We can have a set of values c around 2^{-7} . We can choose:

$$S = \{2^{-10}, 2^{-9}, \dots, 2^{-2}\}$$

(b)

From the testing errors, the best choice of c is 2^{-6} which only has 2.6% misclassification. Larger c produces very small training errors but it overfits (given that we have mislabeled data and somehow the overfit model captured them and hence produces very large testing errors).

c	Training	Testing
2^{-10}	0.0334364 ± 0.0140637	0.0615323 ± 0.0138366
2^{-9}	0.00754235 ± 0.0039516	0.0401613 ± 0.00698532
2^{-8}	$0.000645335 \pm 0.000445091$	0.0312366 ± 0.00494478
2^{-7}	$0.000349556 \pm 0.000586648$	0.0273656 ± 0.00388029
2^{-6}	$0.000275612 \pm 0.000507473$	0.0262634 ± 0.00462889
2^{-5}	$0.000168056 \pm 0.000331714$	0.0311559 ± 0.00454126
2^{-4}	$6.72224e-06 \pm 2.93016e-05$	0.0439247 ± 0.00481205
2^{-3}	0 ± 0	0.0574462 ± 0.00508704
2^{-2}	0 ± 0	0.0655645 ± 0.00419725

Table 1.6. Training Error v.s. Testing Error for Gaussian Kernel

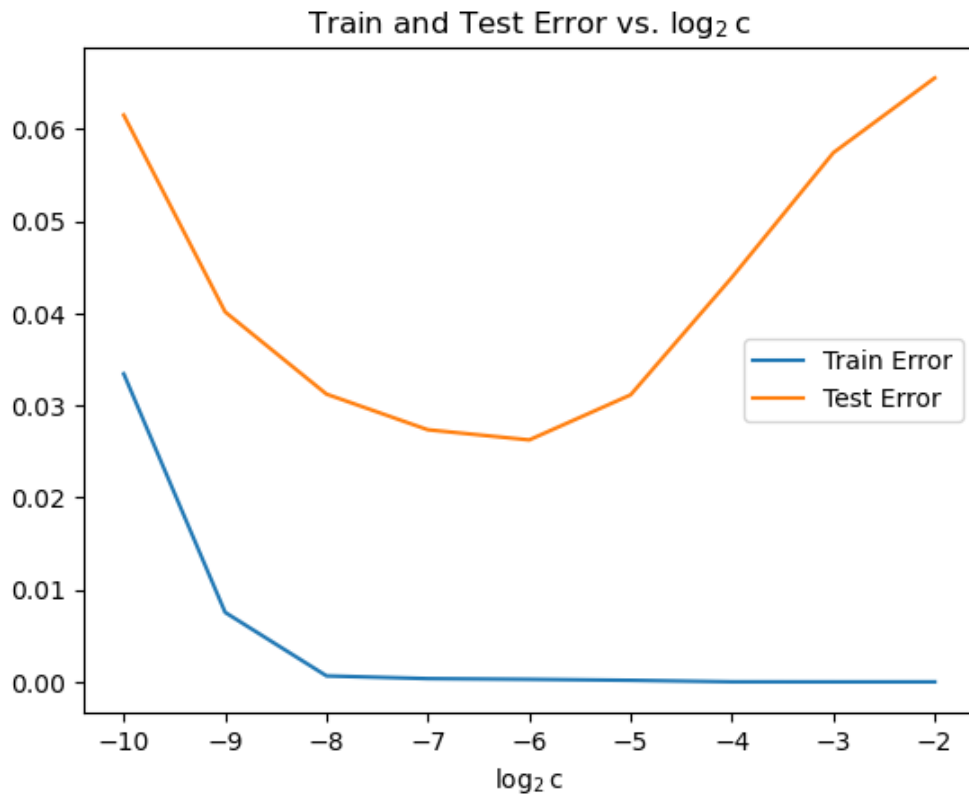


Figure 1.4. Training Error v.s. Testing Error for Gaussian Kernel

(c)

The optimal c^* over 20 runs are:

$$[-6, -6, -6, -6, -7, -6, -7, -6, -6, -6, -6, -6, -6, -6, -6, -7, -6, -7, -6]$$

We have very low testing errors which is 2.6%, lower than polynomial kernel. The best choice of c^* is 2^{-6} with 18 occurrence among 20 runs.

Metric	Mean $\pm$ Std
c^*	0.0140625 $\pm$ 0.003125
Training	0.0001815 $\pm$ 0.0001223
Testing	0.0255376 $\pm$ 0.00365631

(d)

- Polynomial kernels do not overfit the data with higher order. The training errors and testing errors level off with increase of order
- Gaussian kernels, however, overfit the data with larger c . The training errors can be 0 for high c while the testing errors are large
- For optimal choice, polynomial kernel produces similar result as gaussian kernel in terms of testing errors. Gaussian kernel has slightly larger standard deviation.

8

(a)

The One-Versus-One (OvO) algorithm is a method for transforming a binary classifier into a multi-class classifier by constructing a separate binary classifier for every possible pair of classes. Each classifier is trained to distinguish between two specific classes, ignoring the rest. For a dataset with K distinct classes, there are $\frac{K(K-1)}{2}$ unique pairs. For each class pair, we train a separate binary classifier that distinguishes between the two classes. Each classifier is only concerned with correctly classifying instances belonging to its two assigned classes. When predicting the class of a new instance, each binary classifier votes for one of the two classes it was trained on. The final predicted class is the one that receives the majority of votes across all classifiers.

(b)

Since each binary classifier in OvO algorithm only trains corresponding labels. The data is the subset of the full data. The number of epochs needed would be different. We can run several epochs and track how the training error and testing error change over the epochs.

If we run different epochs on binary kernel perceptron classifier with label pair (0, 1) and polynomial order $d = 3$, we can see from the plot that the training error decreases to 0 since

4th epoch and test error levels off. As a side note, even one epoch can achieve very good results, with very few misclassification.

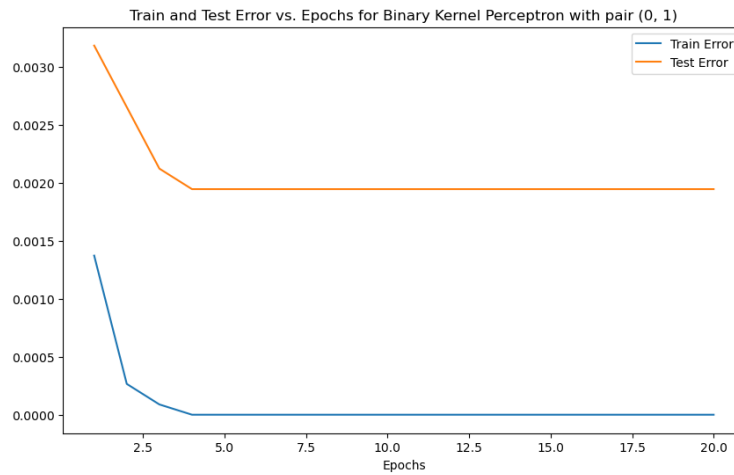


Figure 1.5. Train and Test Error vs. Epochs for Binary Kernel Perceptron with pair (0,1)

If we run different epochs on OvO classifier, we can see the training error is 0 from first epoch. This is because we have majority vote methodology to predict the label and even if there is misclassification in single classifiers, we can still get correct prediction with this ensemble method. For the testing error, it levels off after 6 epochs and does not increase or decrease. We can choose any epochs more than 5 and do not need to worry about overfitting too much.

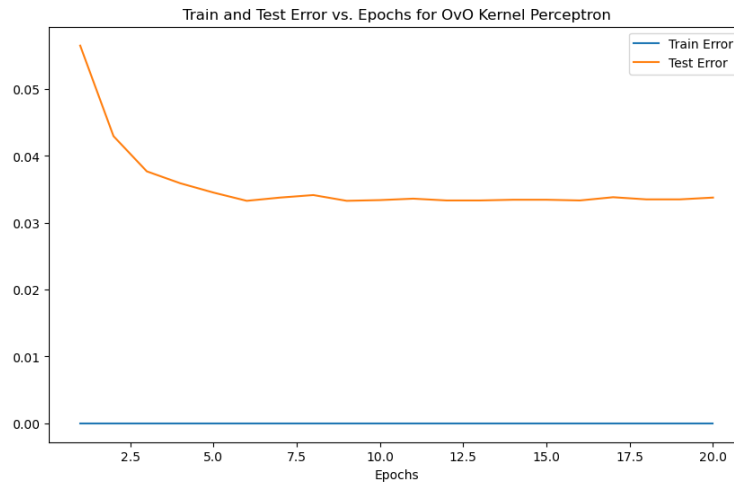


Figure 1.6. Train and Test Error vs. Epochs for OvO Kernel Perceptron

The training errors for all orders of polynomial show 0 while testing error is the smallest on order 3. The testing error is larger than OvR method. Order 4 has slightly larger error rate but lower standard deviation. It is also smaller than any of OvR testing standard deviation, which is the feature of ensemble method - lower variance.

d	Training	Testing
1	0 ± 0	0.0637903 ± 0.00865954
2	0 ± 0	0.0340591 ± 0.00399906
3	0 ± 0	0.0312366 ± 0.00362216
4	0 ± 0	0.031586 ± 0.00275652
5	0 ± 0	0.0329839 ± 0.00338443
6	0 ± 0	0.0340054 ± 0.00366963
7	0 ± 0	0.0343817 ± 0.00347378

Table 1.7. OvO Training Error v.s. Testing Error for Polynomial Kernel

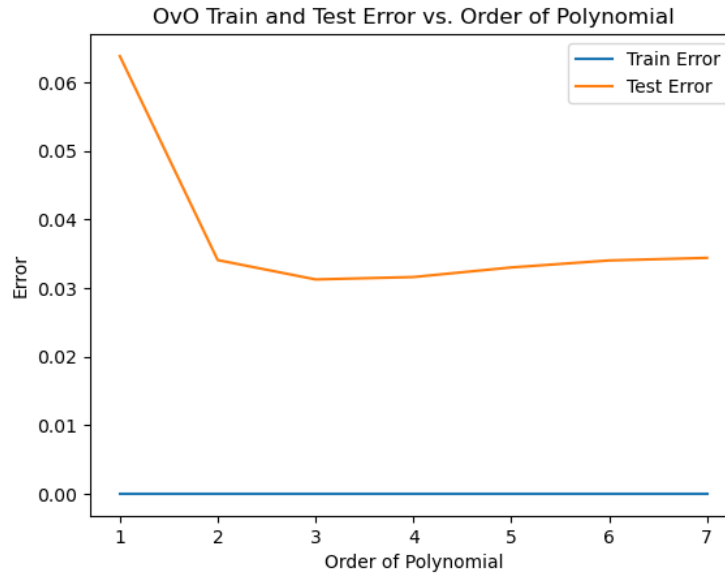


Figure 1.7. OvO Training Error v.s. Testing Error for Polynomial Kernel

(c)

The optimal d^* over 20 runs are:

[4, 4, 4, 6, 3, 4, 3, 3, 3, 4, 4, 5, 3, 3, 5, 2, 3, 6, 5, 3]

We have low testing errors which is 3.2% but higher than polynomial kernel in OvR.

Metric	Mean $\pm$ Std
d^*	3.85 ± 1.061838029
Training	0 ± 0
Testing	0.031586 ± 0.00385404

(d)

- OvO has smaller training errors than OvR. It is 0 for all orders of polynomial kernel, provided with same epochs.
- OvO has higher testing error mean than OvR but smaller testing error standard deviation.
- Training time for OvO is longer than OvR as we need to train $\frac{K(K-1)}{2}$ for OvO compared to K in OvR.